

자료구조의 정석 기초

03 연결 리스트

목차 03 연결 리스트

1. 연결 리스트의 특징
2. 연결 리스트의 연산
3. 연결 리스트의 종류

1. 연결 리스트의 특징



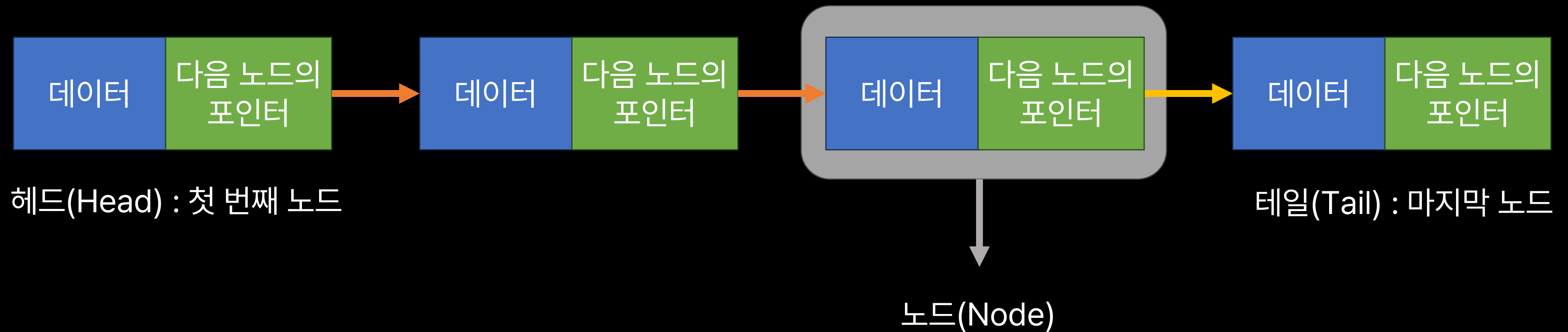
길이 100의 배열을 선언했지만, 실제 사용 데이터는 10개 미만 → 메모리 낭비

길이 100의 배열을 선언했지만, 실제 사용 데이터는 100개 초과 → 메모리 부족

데이터 수에 맞춰 효율적으로 메모리를 활용할 수 있는 자료구조는 없을까?

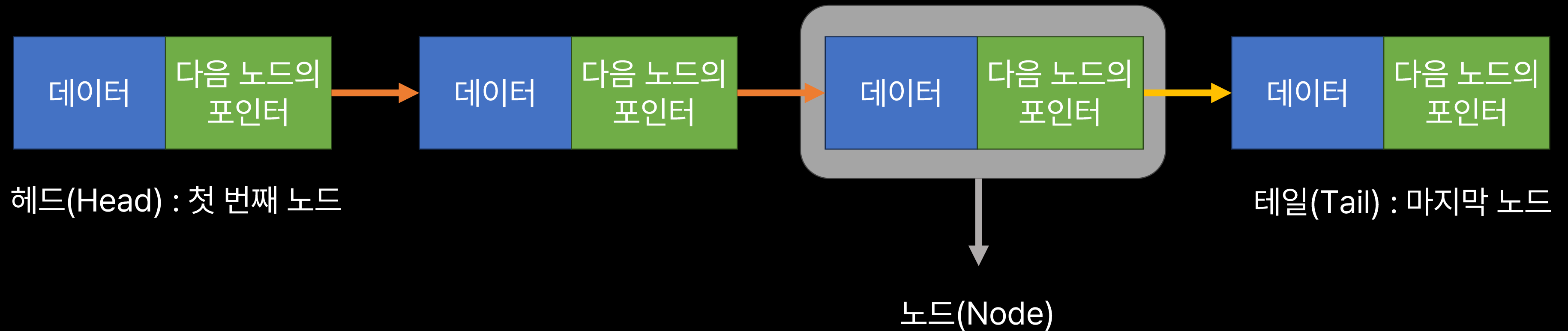
• 연결 리스트(Linked List)란?

- 데이터 목록을 다루는 가장 단순한 자료구조로, 데이터가 **순차적으로 연결된 선형 구조**를 띠며
 - 연결 리스트는 노드(Node)로 구성됨, 노드는 **데이터를 저장하는 부분**과 **다음 노드에 대한 포인터**로 구성
 - 첫 번째 노드를 헤드(Head), 마지막 노드를 테일(Tail)이라 부름



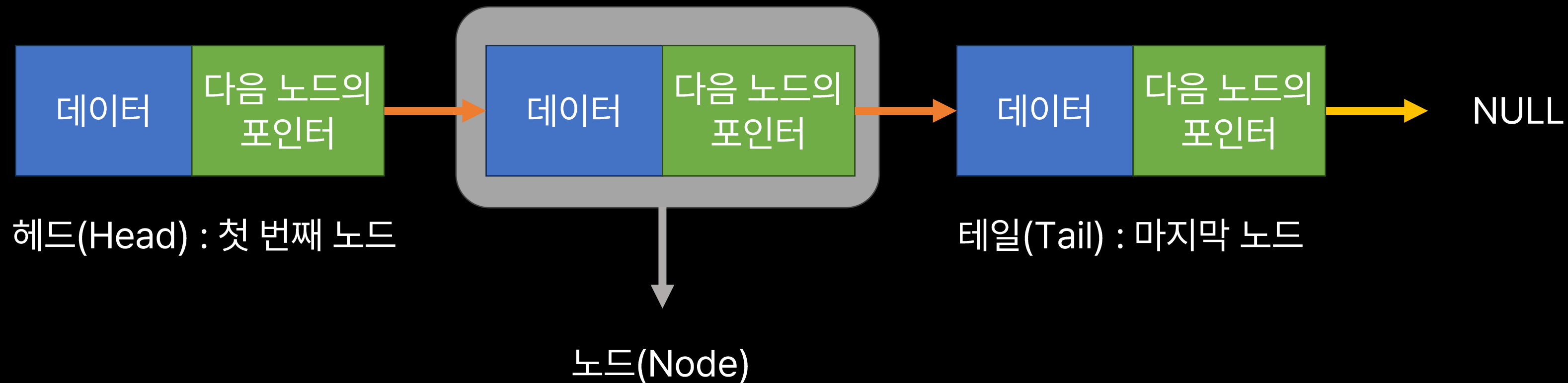
• 연결 리스트(Linked List)의 장점

- 구조가 단순해서 구현이 편하고, 데이터의 추가, 삽입, 삭제가 쉬움
- 다음 노드의 포인터 정보를 통해 추가적인 연산 없이 다음 노드로 접근 가능
- 배열과 달리 메모리 공간을 효율적으로 사용 가능



• 연결 리스트(Linked List)의 단점

- 노드에 다음 노드의 포인터에 대한 정보가 필요하므로 메모리가 추가로 필요
- 인덱스를 통해 접근 가능한 배열과 달리 순차적인 접근만 지원하므로, 특정 요소 탐색 및 업데이트가 오래 걸림

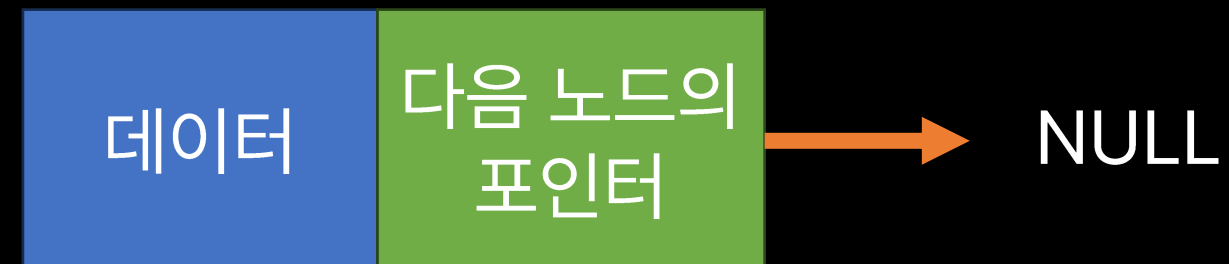


2. 연결 리스트의 연산



• 노드 생성

헤드 노드(테일 노드)



```
// 연결 리스트의 노드 정의
class Node {
public:
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = nullptr;
    }
};
```

C++

```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None
```

Python

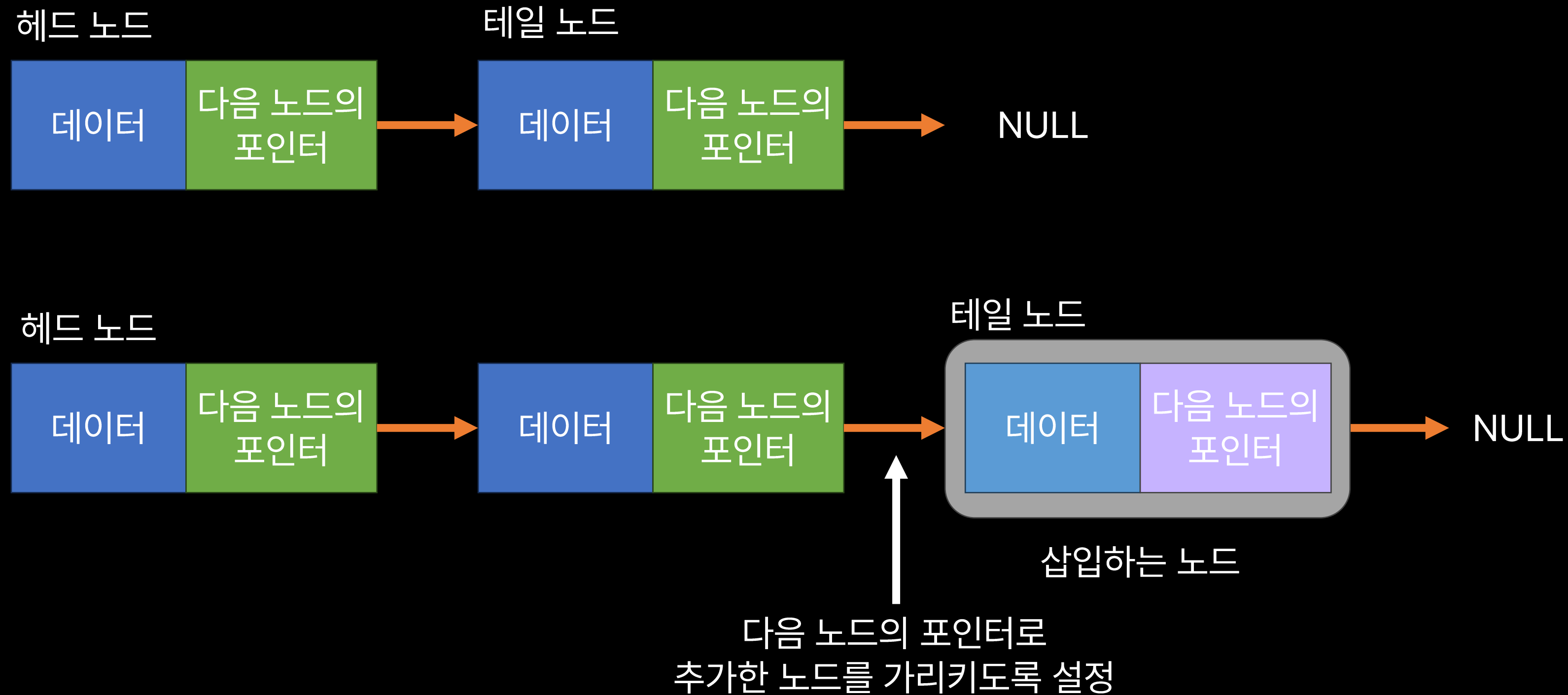
```
class Node {
    int data;
    Node next;

    public Node(int value) {
        data = value;
        next = null;
    }
}
```

Java



• 노드 추가 연산





- 노드 추가 연산

```
// 노드를 맨 끝에 추가
void append(int value) {
    Node* current = head;
    while (current->next != nullptr) {
        current = current->next;
    }
    current->next = new Node(value);
}
```

C++

```
# 노드를 맨 끝에 추가
def append(self, value):
    if not self.head:
        self.head = Node(value)
        return

    current = self.head
    while current.next:
        current = current.next
    current.next = Node(value)
```

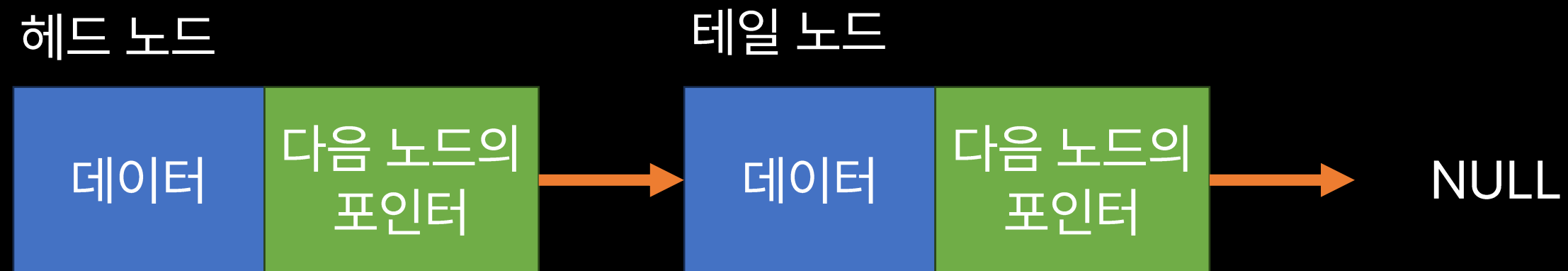
Python

```
// 노드를 맨 끝에 추가
public void append(int value) {
    if (head == null) {
        head = new Node(value);
        return;
    }
    Node current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = new Node(value);
}
```

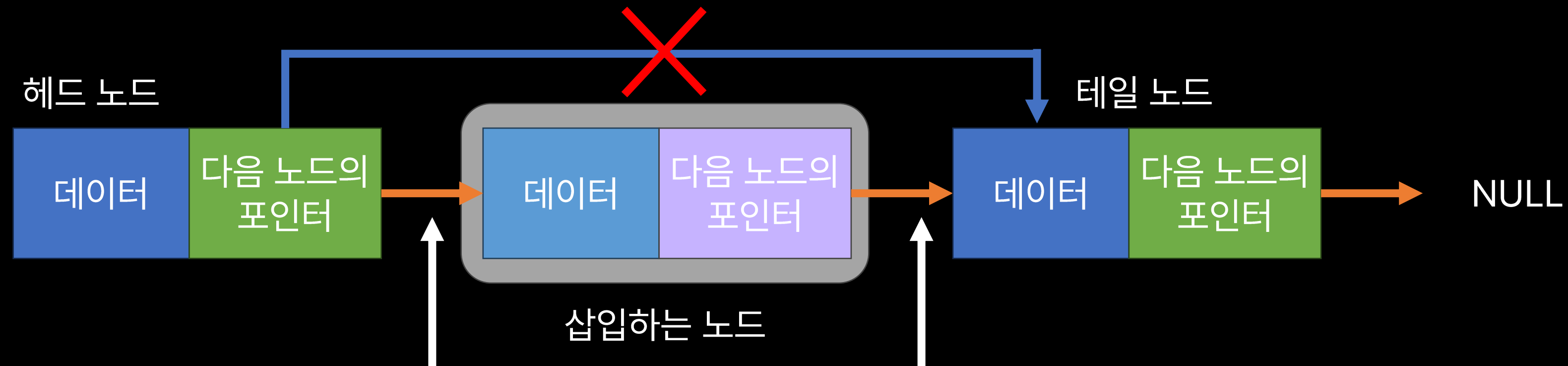
Java



• 노드 삽입 연산



기존에 설정된 포인터를 삽입하는 새로운 노드로 변경





• 노드 삽입 연산

```
// 노드를 특정 위치에 삽입
void insert(int value, int position) {

    Node* newNode = new Node(value);
    Node* current = head;

    for (int i = 0; i < position - 1 && current != nullptr; i++) {
        current = current->next;
    }

    newNode->next = current->next;
    current->next = newNode;
}
```

C++

```
# 노드를 특정 위치에 삽입
def insert(self, value, position):
    new_node = Node(value)

    if position == 0:
        new_node.next = self.head
        self.head = new_node
        return

    current = self.head
    for _ in range(position - 1):
        current = current.next

    new_node.next = current.next
    current.next = new_node
```

Python



- 노드 삽입 연산

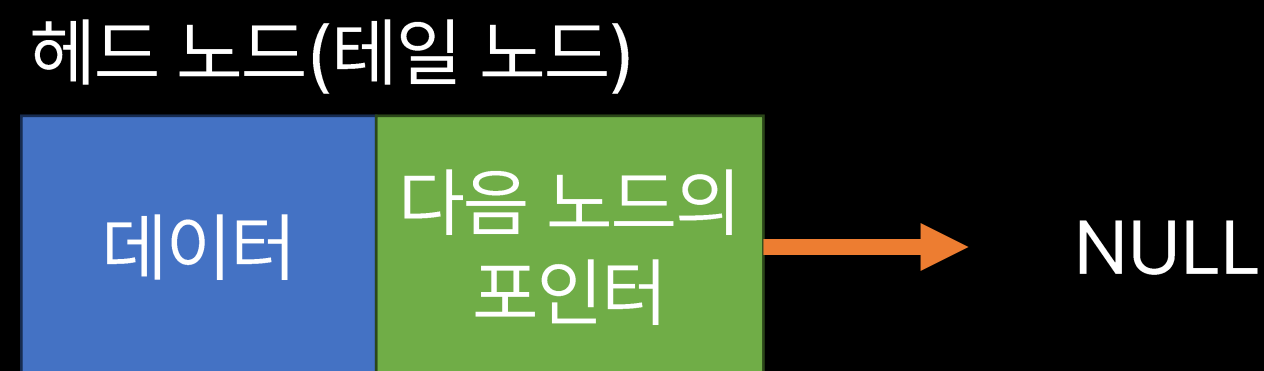
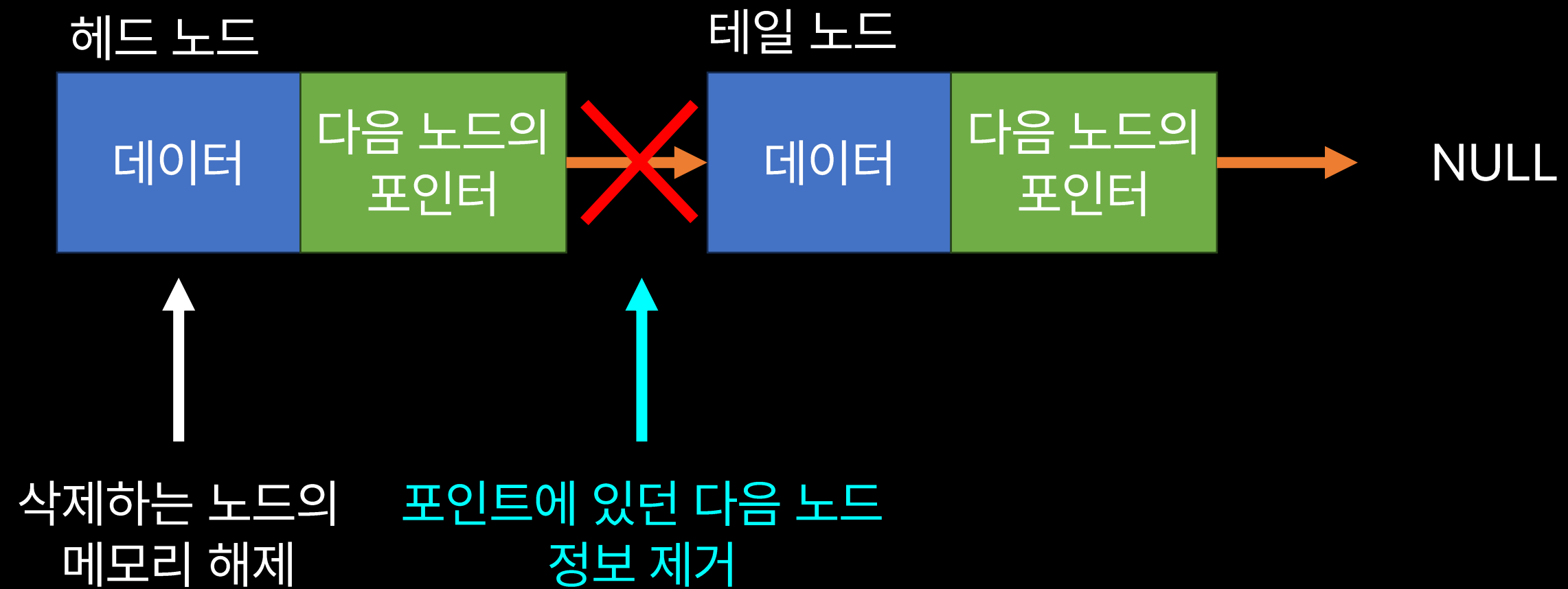
```
// 노드를 특정 위치에 삽입
public void insert(int value, int position) {
    Node newNode = new Node(value);
    if (position == 0) {
        newNode.next = head;
        head = newNode;
        return;
    }
    Node current = head;
    for (int i = 0; i < position - 1 && current != null; i++) {
        current = current.next;
    }

    newNode.next = current.next;
    current.next = newNode;
}
```

Java

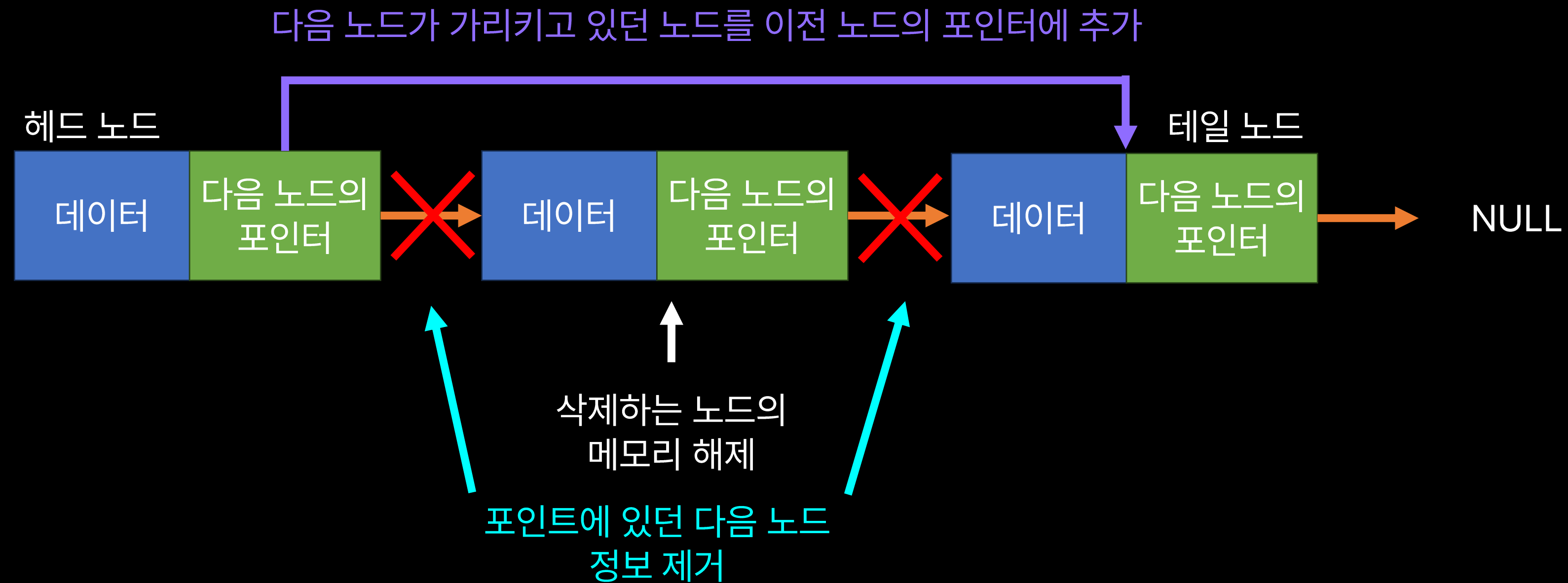


• 헤드 노드 삭제 연산



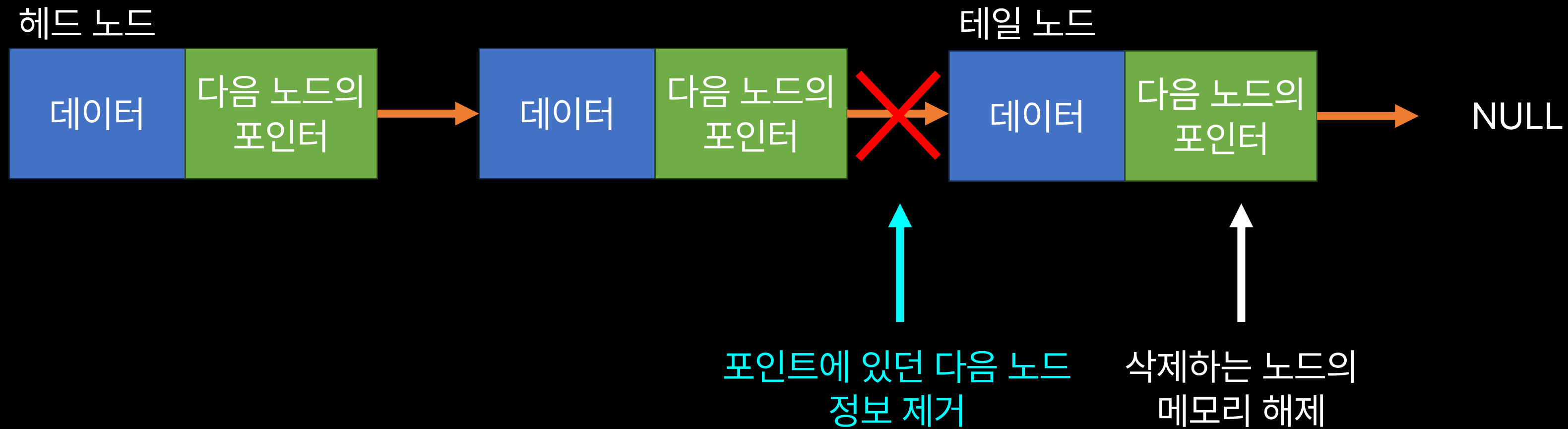


• 중간 노드 삭제 연산





• 테일 노드 삭제 연산





• 노드 삭제 연산

```
// 노드 삭제
void remove(int value) {

    if (head->data == value) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }

    Node* current = head;
    while (current->next != nullptr && current->next->data != value) {
        current = current->next;
    }

    Node* temp = current->next;
    current->next = current->next->next;
    delete temp;
}
```

C++

```
# 노드 삭제
def remove(self, value):

    if self.head.data == value:
        self.head = self.head.next
        return

    current = self.head
    while current.next and current.next.data != value:
        current = current.next

    current.next = current.next.next
```

Python



- 노드 삭제 연산

```
// 노드 삭제
public void remove(int value) {

    if (head.data == value) {
        head = head.next;
        return;
    }

    Node current = head;
    while (current.next != null && current.next.data != value) {
        current = current.next;
    }

    current.next = current.next.next;
}
```

Java

3. 연결 리스트의 종류

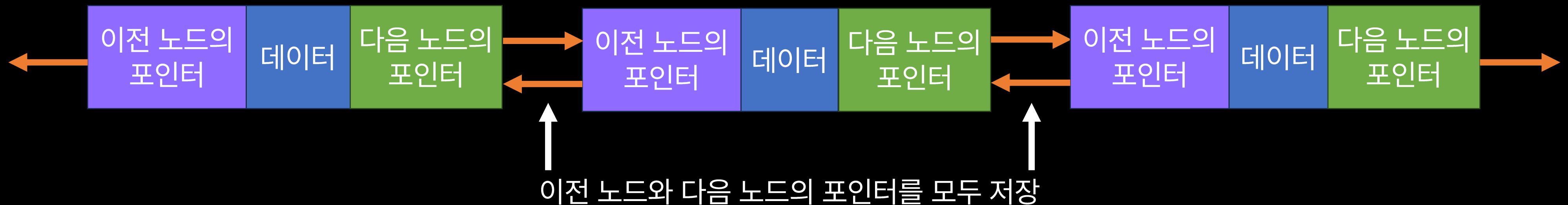


- 접근(Access) 시간 복잡도 : $O(N)$
 - 인덱스 N 에 접근하기 위해, 헤드 노드에서 다음 노드로 N 번 이동
 - 배열의 경우에는 특정 인덱스에 바로 접근 가능 $\rightarrow O(1)$
- 삽입/삭제 시간 복잡도 : $O(1)$
 - 삽입, 삭제할 노드의 주변 노드만 수정하면 됨 $\rightarrow$ 소요 시간이 특정 값에 비례하지 않고 항상 일정
 - 배열의 경우에는 특정 인덱스에 데이터를 삽입, 삭제하기 위해 기존 데이터를 옮기는 작업이 필요 $\rightarrow O(N)$



• 이중 연결 리스트(Doubly Linked List)

- 특정 노드를 탐색해야 할 때 단방향으로 이동해야 하는 단방향 연결 리스트의 단점 해결 가능
- 삽입 및 삭제 연산이 빠름
- 단, 포인터를 위한 메모리 공간이 2배로 사용되며, 구현이 상대적으로 복잡





• 원형 연결 리스트(Doubly Linked List)

- 순환 구조로 인한 유연성으로 인해 리스트의 끝에서 시작으로 쉽게 이동할 수 있음
- 구현이 복잡하고 메모리 공간이 추가로 필요

